

University Name: **R. P. Shaha University**
Team Name: **RPSU_Ternary_Troopers**
Team No: **180**
Team ID: **1111825**
Building/Room No: **03/504**

Coach: **ANKAN RAY**
Contestant 1: **SIAM HOSSAIN**
Contestant 2: **TYIBOR RAHMAN**
Contestant 3: **MD JISAN**

Level 01:

```
typedef long long ll;
typedef vector<int> vi;
typedef pair<int, int> pii;
#define nline "\n"
#define Yes cout << "YES\n"
#define No cout << "NO\n"
#define all(x) (x).begin(), (x).end()
#define sz(x) (int)(x).size()
#define pb push_back
#define ff first
#define ss second
#define MOD 1000000007
#define fraction() cout.unsetf(ios::floatfield); cout.precision(10); cout.setf(ios::fixed,ios::floatfield);
int dx[] = {0, 0, +1, -1};
int dy[] = {+1, -1, 0, 0};
const double eps = 1e-9;
ios::sync_with_stdio(0);
cin.tie(0);
```

bool is_prime(ll n)

```
{
    if (n <= 1)
        return 0;
    else if (n == 2 or n == 3)
        return 1;
    else if (n % 2 == 0)
        return 0;
    else
        for (ll b = 3; b * b <= n; b += 2)
        {
            if (n % b == 0)
                return 0;
        }
    return 1;
}
```

string dtob(int n)

```
{
    string a = "";
    while (n)
        a += (n & 1) + '0', n >>= 1;
}
```

```

        reverse(a.begin(), a.end());
        return a;
    }
void inverse(string &a)
{
    for (char &ch : a)
        ch = '1' - ch + '0';
}
ll btod(string a)
{
    ll n = a.size(), res = 0;
    for (int b = 0; b < n; b++)
    {
        res += (a[b] - '0') * (1 << (n - b - 1));
    }
    return res;
}
ll count1(int n)
{
    ll cnt = 0;
    while (n)
    {
        if (n & 1)
            cnt++;
        n >>= 1;
    }
    return cnt;
}
ll get(ll n, ll k)
{
    return ((n >> (k - 1)) & 1);
}
vector<ll> get_devisors(ll n)
{
    vector<ll> div;
    for (ll b = 1; b * b <= n; b++)
    {
        if (n % b == 0)
        {
            div.push_back(b);
            if (b * b != n)
                div.push_back(n / b);
        }
    }
}

```

```

    return div;
}
void prime_fact(ll n, map<ll, ll> &mp)
{
    while (n % 2 == 0)
        mp[2]++, n /= 2;
    for (ll b = 3; b * b <= n; b += 2)
    {
        while (n % b == 0)
            mp[b]++, n /= b;
    }
    if (n > 1)
        mp[n]++;
}
vector<bool> primes;
void seive(ll n)
{
    primes.resize(n + 1, true);
    primes[0] = primes[1] = false;
    for (ll c = 4; c <= n; c += 2)
        primes[c] = false;
    for (ll b = 3; b * b <= n; b += 2)
    {
        if (primes[b])
        {
            for (ll c = b * b; c <= n; c += b)
                primes[c] = false;
        }
    }
}
ll nCr(ll n, ll r)
{
    if (r > n)
        return 0;
    if (r == 0 || r == n)
        return 1;
    ll res = 1;
    for (ll i = 1; i <= r; ++i)
        res *= (n - r + i) / i;
    return res;
}
ll lcm(ll a, ll b)
{
    return a / __gcd(a, b) * b;
}

```

```

}
void set_bit_1(ll &n, ll k)
{
    n = n | (1 << k - 1);
}
void reset_bit(ll &n, ll k)
{
    int s = 1 << (k - 1);
    n &= ~s;
}
bool is_pow_of_two(ll n)
{
    if (n <= 0)
        return 0;
    return !(n & (n - 1));
}
vector<ll> spf(1e6 + 5, 1);
void prime_fact_with_seive()
{
    for (ll b = 2; b <= 1e6; b++)
    {
        if (spf[b] == 1)
        {
            for (int c = b; c <= 1e6; c += b)
                spf[c] = b;
        }
    }
}
void get_prime_fact(ll n, map<ll, ll> &mp)
{
    while (n != 1)
    {
        mp[spf[n]]++;
        n /= spf[n];
    }
}
ll fast_pow(ll base, ll power, ll mod = LLONG_MAX)
{
    if (power == 0)
        return 1;
    ll res = 1, n = base;
    while (power)
    {
        if (power & 1)

```

```

        res *= base, power--;
    if ((power & 1) == 0)
        power /= 2, base *= base;
    res %= mod;
    base %= mod;
}
return res;
}
ll sum_digits(ll a)
{
    ll s = 0;
    while (a)
        s += a % 10, a /= 10;
    return s;
}
int factorial(int a)
{
    int f;
    if (a == 0 || a == 1) return 1;
    f = a * factorial(a - 1);
    return f;
}
int Extended_gcd(int a, int b, int &x, int &y)
{
    if (b == 0)
    {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = Extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
} // ax + by = gcd(a,b)
string str = "12";
do
{
    } while (next_permutation(all(str)));

```

Level_02:

bool isBalances(char top, char ch)

```
{
    if (top == '(' && ch == ')' || top == '{' && ch == '}' || top == '[' && ch == ']')
        return 1;
    else
        return 0;
}
```

int main()

```
{
    string s; // [{()}]
    bool ans = true;
    stack<char> st;
    for (char ch : s)
    {
        if (ch == '(' || ch == '{' || ch == '[')
            st.push(ch);
        else
        {
            if (st.empty())
            {
                ans = false;
                break;
            }
            else
            {
                if (isBalances(st.top(), ch))
                    st.pop();
                else
                {
                    ans = false;
                    break;
                }
            }
        }
    }
}
```

void Kadaness()

```
{
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++)
```

```

{
    cin >> arr[i];
}
int currentSum = arr[0];
int maxSum = arr[0];
for (int i = 1; i < n; i++)
{
    currentSum = max(arr[i], arr[i] + currentSum);
    maxSum = max(maxSum, currentSum);
}
cout << maxSum << endl;
}
bool cmp(const pair<int, int> &p1, const pair<int, int> &p2)
{
    if(p1.first > p2.first) return 1;
    else if(p1.first == p2.first) return (p1.second < p2.second);
    return 0;
}

```

Level-03:

set<int> Divisors(int n)

```

{
    set<int> multiplicand;
    for (int i = 1; i <= n; i++)
    {
        multiplicand.insert(__gcd(i, n));
    }
    return multiplicand;
}

```

ll EulerPhi(ll n) // gcd(a,N) = d ; How many gcd(a,N) = d where d divides N

```

{
    // solution : Phi(N/d);
    ll phi = n;
    // when we divide two numbers with their gcd then they are coprime
    for (auto it : primes) // gcd(9/3,12/3) = gcd(3,4) = 1

```

```

{
    if (1LL * it * it > n)
        break;
    if (n % it == 0)
    {
        while (n % it == 0)
        {
            n /= it;
        }
        phi /= it;
    }
}

```

```

        phi *= (it - 1);
    }
}
if (n > 1)
{
    phi /= n;
    phi *= (n-1);
}
return phi;
}
int phi[mx];
int Sum_Coprime(int n)
{
    seive(limit);
    for (int i = 1; i <= limit; i++)
    {
        phi[i] = i;
    }
    for (auto p : primes)
    {
        for (int i = p; i <= limit; i += p)
        {
            phi[i] /= p;
            phi[i] *= (p - 1);
        }
    }
    ll phi_sum = (1LL * phi[n] * n) / 2;
    return phi_sum;
}

```


Level_04:

void dfs(int u) // using recursion

```
{
    isVisited[u] = 1;
    cout << u << ' ';
    for (auto v : adj[u])
    {
        if (!isVisited[v])
        {
            dfs(v);
        }
    }
    return;
}
```

void dfs(int s) // using stack

```
{
    vis[s] = 1;
    stack<int> q;
    q.push(s);
    while (!q.empty())
    {
        int u = q.top();
        q.pop();
        cout << u << ' ';
        for (auto v : adj[u])
        {
            if (vis[v] == 0)
            {
                vis[v] = 1;
                q.push(v);
            }
        }
    }
}
```

void bfs(int s)

```
{
    memset(level, -1, sizeof(level));
    level[s] = 0;
    queue<int> q;
    q.push(s);
    while (!q.empty())
    {
        int u = q.front();
        q.pop();
```

```

        for (auto v : adj[u])
        {
            if (level[v] == -1)
            {
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }
}

int Coin_Change(vector<int> &coins, int amounts)
{
    if (amounts == 0)
        return 0;
    if (dp[amounts] != -1)
        return dp[amounts];
    ll ans = INT_MAX;
    for (int coin : coins)
    {
        if (amounts - coin >= 0)
            ans = min(ans, func(coins, amounts - coin) + 1ll);
    }
    return dp[amounts] = ans;
}

int longestCommonSubsequence(int i, int j, string &s1, string &s2)
{
    if (i < 0 || j < 0)
        return 0;
    if (dp[i][j] != -1)
        return dp[i][j];
    int ans = solve(i - 1, j, s1, s2);
    ans = max(ans, solve(i, j - 1, s1, s2));
    ans = max(ans, solve(i - 1, j - 1, s1, s2) + (s1[i] == s2[j]));
    return dp[i][j] = ans;
}

bool Equal_Subset_Sum(int i, int sum, vector<int> &num)
{
    if (sum == 0)
        return true;
    if (i < 0)
        return false;
    if (dp[i][sum] != -1)
        return dp[i][sum];
    bool flag = solve(i - 1, sum, num);

```

```

    if (sum - num[i] >= 0)
        flag |= solve(i - 1, sum - num[i], num);
    return dp[i][sum] = flag;
}

int Frog_Jump(int n)
{
    if (n == 1)
        return 0;
    if (dp[n] != -1)
        return dp[n];
    int dis1 = INT_MAX;
    int dis2 = INT_MAX;
    dis1 = func(n - 1) + abs(h[n] - h[n - 1]);
    if (n > 2)
        dis2 = func(n - 2) + abs(h[n] - h[n - 2]);
    dp[n] = min(dis1, dis2);
    return dp[n];
}

long long Knapsack(int i, int currW) // Based on weight
{
    if (i > N)
        return 0;
    if (dp[i][currW] != -1)
        return dp[i][currW];
    long long ret1 = 0, ret2 = 0;
    if (w[i] + currW <= W)
        ret1 = v[i] + solve(i + 1, w[i] + currW);
    ret2 = 0 + solve(i + 1, currW);
    return dp[i][currW] = max(ret1, ret2);
}

void reconstruct(int i, int currW) // For the knapsack path
{
    if (i > n)
        return;
    int best = solve(i, currW);
    if (currW + w[i] <= sz && best == p[i] + solve(i + 1, currW + w[i]))
    {
        cout << i << " ";
        reconstruct(i + 1, currW + w[i]);
    }
    else
    {
        reconstruct(i + 1, currW);
    }
}

```

```

    }
}
long long solve(int i, long long cp) // Based on price
{
    if (cp == 0)
        return 0;
    if (i < 1)
        return 1e15;
    if (dp[i][cp] != -1)
        return dp[i][cp];
    long long ans = solve(i - 1, cp);
    if (cp - p[i] >= 0)
        ans = min(ans, w[i] + solve(i - 1, cp - p[i]));
    return dp[i][cp] = ans;
}
int func(vector<int> &arr, int n) // Longest Increasing Subsequence
{
    if(dp[n] != -1) return dp[n];
    int cnt = 1;
    for (int i = 0; i < n; i++) {
        if (arr[i] < arr[n]) {
            cnt = max(cnt, func(arr, i) + 1);
        }
    }
    dp[n] = cnt;
    return dp[n];
}
int solve(int len, vector<int> &prices) // Rod Cutting
{
    if (len == 0)
        return 0;
    if (dp[len] != -1)
        return dp[len];
    int mx = -1;
    for (int i = 1; i <= n; i++)
    {
        if (len - i >= 0)
            mx = max(mx, solve(len - i, prices) + prices[i - 1]);
    }
    return dp[len] = mx;
}

```

Level_05:

Dsu {

```
int find_par(int u)
{
    if (par[u] == u)
        return u;
    return find_par(par[u]);
}
void Union(int u, int v)
{
    par[v] = u;
}
void Union(int u, int v) // more optimal and sz[i<=n] = 1 initially
{
    if (sz[u] < sz[v])
        swap(u, v);
    par[v] = u;
    sz[u] += sz[v];
}
int uR = find_par(u);
int vR = find_par(v);
if (uR != vR)
    Union(u, v);
}
```

Dijkstra{

```
int findMinDistance(int distance[], int visited[])
{
    int min = INF, MinIndex;
    for (int i = 0; i < V; i++)
    {
        if (!visited[i] && distance[i] <= min)
        {
            min = distance[i];
            MinIndex = i;
        }
    }
    return MinIndex;
}
void diakstra(int graph[V][V], int start)
{
    int distence[V];
    int visited[V];
}
```

```

for (int i = 0; i < V; i++)
{
    distance[i] = INF;
    visited[i] = 0;
}
distance[start] = 0;
for (int count = 0; count < V - 1; count++)
{
    int u = findMinDistance(distance, visited);
    visited[u] = 1;
    for (int v = 0; v < V; v++)
    {
        if (!visited[v] && graph[u][v] != INF && distance[u] != INF && distance[u] + graph[u][v] <
distance[v])
        {
            distance[v] = distance[u] + graph[u][v];
        }
    }
}
for (int i = 0; i < V; i++)
{
    if (distance[i] == INF)
    {
        cout << i << "\tINF" << endl;
    }
    else
        cout << i << "\t" << distance[i] << endl;
}
}

}

```

```

void bellman(int graph[N][N])
{
    dist[0] = 0;
    for (int k = 0; k < N - 1; k++)
    {
        for (int i = 0; i < N; i++)
        {
            for (int j = i+1; j < N; j++)
            {
                if (graph[i][j] == INF)

```

```

        continue;
        if (dist[i] + graph[i][j] < dist[j])
            dist[j] = dist[i] + graph[i][j];
    }
}
}
for (int it : dist)
    cout << it << ' ';
cout << endl;
}

```

void floydWarshall(vector<vector<int>> &dist, int V)

```

{
    for (int k = 0; k < V; k++)
    {
        for (int i = 0; i < V; i++)
        {
            for (int j = 0; j < V; j++)
            {
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
            }
        }
    }
}

```

Primes algo {

```

void printMst(int parent[], int graph[V][V]) {
    cout << "Edge \tWeight\n";
    for (int i = 1; i < V; i++) {
        parent[0] == -1;
        cout << parent[i] + 1 << " - " << i + 1 << "\t" << graph[i][parent[i]] << endl;
    }
}

```

int MinWeightVertex(int key[], int mstSet[]) {

```

    int mini = INF, Vertex = -1;
    for (int i = 0; i < V; i++) {
        if (!mstSet[i] && key[i] < mini) {
            mini = key[i];
            Vertex = i;
        }
    }
    return Vertex;
}

```

```

void Prime(int graph[V][V]) {
    int mstSet[V];
    int parent[V];
    int key[V];

    for (int i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = MinWeightVertex(key, mstSet);
        mstSet[u] = true;

        for (int v = 0; v < V; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }
    printMst(parent, graph);
}

void MultiStage_shortestDist(int graph[N][N], int src)
{
    int distance[N];
    distance[N - 1] = 0;
    for (int i = N - 2; i >= 0; i--)
    {
        distance[i] = INF;
        for (int j = i; j < N; j++)
        {
            if (graph[i][j] == INF)
                continue;
            distance[i] = min(distance[i], graph[i][j] +
                             distance[j]);
        }
    }
    cout << "Minimum distance to visit node 7 is: " << distance[src] << endl;
}

```


Number Theory Facts:

1. Contiguous Integers:

The expression $n(n+1)(n+2)(n+3) + 1$ results in a perfect square. This outcome is based on the product of four consecutive integers and holds true for specific values of n .

2. Perfect Squares:

For any positive integer n , the difference $n^2 - (n-1)^2 = 2n - 1$ represents an odd number. That is, the difference between the squares of two consecutive integers is always an odd number.

3. Sum of First n Natural Numbers:

The formula $1 + 2 + 3 + \dots + n = n(n+1)/2$ denotes the sum of the first n natural numbers and can also be interpreted geometrically as the area of a trapezoid.

4. Fermat's Little Theorem:

If p is a prime number and a is an integer not divisible by p , then $a^{(p-1)} \equiv 1 \pmod{p}$.

5. Divisibility Rules:

Divisibility rules for various numbers are very useful.

For example, a number is divisible by 3 if the sum of its digits is divisible by 3.

And, A Number is Divisible by 9 if the Sum of its Digits is Divisible by 9

6. Goldbach's Conjecture:

According to this conjecture, every even integer $n \geq 2$ can be expressed as the sum of two prime numbers.

7. Congruences:

The expression $a \equiv b \pmod{m}$ means that a and b leave the same remainder when divided by m . This concept plays a significant role in solving various number-theoretic problems.

8. Chinese Remainder Theorem:

If $m_1, m_2, \dots, m_k$ are pairwise coprime positive integers and $a_1, a_2, \dots, a_k$ are any integers, then the system of congruences $x \equiv a_i \pmod{m_i}$ has a unique solution modulo $M = m_1 * m_2 * \dots * m_k$.

9. Sum of Divisors Function:

The function $\sigma(n)$ denotes the sum of all positive divisors of a number n , and it plays an important role in various mathematical problems.

10. Euler's Totient Function:

Euler's Totient Function is used to count the number of integers less than or equal to n that are relatively prime to n . It is particularly significant in number theory and cryptography.

11. Prime Numbers:-

A prime Number is a number that has exactly two factors- 1 and itself.

EX: 2, 3, 5, 7, 17, etc.

Fun fact: The product of two prime numbers always has 4 divisors.

Example:

6 --> 1, 2, 3, 6

15 --> 1, 3, 5, 15

12. Armstrong Number:-

An Armstrong number is a number that is equal to the sum of its digits raised to the power of the number of digits.

EX: 1,

$$153 = 1^3 + 5^3 + 3^3,$$

$$1634 = 1^4 + 6^4 + 3^4 + 4^4$$

12. If any number has only three positive divisors, then the number has to be the perfect square of a prime number.

EX: $4 = 2^2$; Divisors of 4 = 1, 2, 4

$25 = 5^2$; Divisors of 25 = 1, 5, 25

13. Palindrome Number:-

A palindromic number is a number that is the same as its reverse form.

EX: 123321, 1111, 1331, etc.

14. Digital Root:-

The digital root of a number is the single-digit value we get by repeatedly adding the digits of a number until we reach a single digit.

It's also known as the digital sum or seed number.

EX: $221 \rightarrow 2 + 2 + 1 = 5$

$698 \rightarrow 6 + 9 + 8 = 23 \rightarrow 2 + 3 = 5$

Fun fact:- DR = remainder when the number is divided by 9 (Number % 9). [If number % 9 = 0 and number $\neq$ 0, then DR = 9.]

15. Simplest but important rules of number theory:-

- Perfect Squares Have an Odd Number of Total Divisors.
- Any Two Consecutive Numbers are Coprime. [$\gcd(n, n+1)=1$]
- The Product of Two Consecutive Integers is Never a Perfect Square.
- Any Power of an Odd Number is Odd. [$\text{odd}^n = \text{odd}$]

16. Some important Progression-

$$1+2+3+4+\dots+n=\{n(n+1)\}/2$$

$$1^2+2^2+3^2+4^2+\dots+n^2=\{n(n+1)(2n+1)\}/6$$

$$1^3+2^3+3^3+4^3+\dots+n^3=[\{n(n+1)\}/2]^2$$

$$\text{sum of first } n \text{ odd} = n^2$$

$$\text{sum of first } n \text{ even} = n(n+1)$$

17. Arithmetic progression

$$a+(a+r)+(a+2r)+(a+3r)+\dots$$

a->starting value

r->progress rate(2nd term-first term)

$$\text{nth term} = a + (n-1)r$$

$$\text{sum of first n value} = [n\{2a + (n-1)r\}]/2$$

18. Geometric progression

$$a+ar+ar^2+ar^3+\dots$$

a->starting value

r->progress rate(second term/first term)

$$\text{nth term} = ar^{(n-1)}$$

$$\text{if}(r \geq 1), \text{Sum of first n values} = \{a(r^n - 1)\}/(r - 1)$$

$$\text{if}(r < 1), \text{Sum of first n values} = \{a(1 - r^n)\}/(1 - r)$$

• POINTS

Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Midpoint

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2} \right)$$

Section (m:n)

$$\left(\frac{mx_2 + nx_1}{m+n}, \frac{my_2 + ny_1}{m+n} \right)$$

• STRAIGHT LINE

Slope

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

Line equation

$$ax + by + c = 0$$

Two-point form

$$\frac{y - y_1}{y_2 - y_1} = \frac{x - x_1}{x_2 - x_1}$$

Point-line distance

$$d = \frac{|ax_1 + by_1 + c|}{\sqrt{a^2 + b^2}}$$

Angle between lines

$$\tan \theta = \left| \frac{m_1 - m_2}{1 + m_1 m_2} \right|$$

Parallel

$$m_1 = m_2$$

Perpendicular

$$m_1 m_2 = -1$$

♦ TRIANGLE

Area (coordinates)

$$\frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

Heron

$$s = \frac{a+b+c}{2} \quad A = \sqrt{s(s-a)(s-b)(s-c)}$$

Centroid

$$\left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

♦ VECTOR

Dot product

$$\vec{a} \cdot \vec{b} = |a||b| \cos \theta$$

Cross product (2D)

$$a_x b_y - a_y b_x$$

Orientation

$$(B - A) \times (C - A)$$

♦ POLYGON

Shoelace area

$$\frac{1}{2} \left| \sum (x_i y_{i+1} - y_i x_{i+1}) \right|$$

Permutations of 123

♦ ANGLES

Angle between vectors

$$\theta = \cos^{-1} \left(\frac{a \cdot b}{|a||b|} \right)$$

Degree → Radian

$$\text{rad} = \text{deg} \times \frac{\pi}{180}$$

♦ LINE SEGMENTS

Projection of point P on AB

$$\frac{(P - A) \cdot (B - A)}{|B - A|^2}$$

Distance point → segment

$$\min(\text{dist to projection or endpoints})$$

♦ COLLINEARITY

$$\text{Area} = 0$$

- CONVEX HULL

Turn test

$$(b-a) \times (c-a)$$

- NUMERICAL

EPS

$$|a-b| < 10^{-9}$$

- PARABOLA

Permutations of 123

$$y = ax^2 + bx + c$$

Vertex

$$x = -\frac{b}{2a}$$

- ELLIPSE

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

- HYPERBOLA

$$\frac{x^2}{a^2} - \frac{y^2}{b^2} = 1$$